

Práctica 2: Problema del Productor-Consumidor con Semáforos y Mutex

Torres Corte Aarón Ulises

Introducción al Cómputo de Alto Rendimiento
Facultad de Ciencias de la Computación, BUAP

Resumen En este reporte se documenta e inspecciona a profundidad la implementación de la Práctica 2, la cual tiene como objetivo resolver el clásico problema de sincronización y concurrencia conocido como el *Productor-Consumidor* (también llamado problema del búfer limitado). En el desarrollo de esta solución se emplea una arquitectura basada en una cola circular compartida en memoria, combinada con el uso intensivo de hilos de ejecución POSIX (`pthread`). Para asegurar la integridad de los datos, la solución hace uso de mecanismos de sincronización fundamentales en la teoría de sistemas operativos: semáforos contadores (`sem_t`) para la señalización de disponibilidad, y bloqueos de exclusión mutua (`pthread_mutex_t`) para la protección de la sección crítica. El documento ahonda en la teoría subyacente de estos mecanismos, explicando su funcionamiento interno y su criticidad matemática. Adicionalmente, se proporciona un análisis del comportamiento empírico del programa durante su ejecución, centrando la atención en la asimetría de procesamiento observada entre los hilos consumidores.

Keywords: Concurrencia · Productor-Consumidor · Semáforos · Mutex · Hilos POSIX · Sistemas Operativos

1. Introducción y Marco Teórico

El problema del Productor-Consumidor, formalizado originalmente por Edsger W. Dijkstra en 1965, es un paradigma central en el diseño de sistemas operativos y en la programación concurrente. Este problema ilustra los desafíos inherentes a la sincronización de múltiples hilos de ejecución que requieren acceso coordinado a un recurso de memoria compartido.

En el escenario propuesto, una entidad denominada *Productor* genera datos y los deposita en el búfer. Simultáneamente, entidades *Consumidoras* extraen y procesan dicha información. Dado que la planificación de hilos es controlada de forma asíncrona por el sistema operativo, las instrucciones de lectura y escritura pueden entrelazarse de maneras impredecibles (fenómeno conocido como entrelazado o *interleaving*). Sin los controles adecuados, esto derivaría invariablemente en fallas catastróficas, tales como la sobrescritura de datos útiles, la lectura de segmentos vacíos, o la corrupción de los índices de la estructura (condiciones de carrera o *race conditions*).

Para solucionar este problema se requiere del uso combinado de dos herramientas de sincronización vitales: los Semáforos y los Mutex.

2. Mecanismos de Sincronización: Semáforos y Mutex

Aunque a menudo se confunden en su uso práctico, los semáforos y los mutex tienen propósitos matemáticos y semánticos fundamentalmente distintos en el diseño de sistemas concurrentes. A continuación, se detalla su funcionamiento interno y su rol en la solución implementada.

2.1. El Mutex (Exclusión Mutua)

Un **mutex** (del inglés *Mutual Exclusion*) es un mecanismo de cerrojo diseñado específicamente para proteger las **secciones críticas** del código; es decir, aquellas líneas donde se modifican variables compartidas.

Para garantizar que nadie más entre, la adquisición de un mutex ocurre como un evento atómico, es decir, una operación completamente indivisible. Si el cerrojo ya está ocupado por otro hilo, el sistema operativo interviene de inmediato y pone a dormir (bloquea) al hilo solicitante en una cola de espera, evitando que se quede en un bucle consumiendo ciclos de procesador inútilmente.

El concepto más importante de un mutex es la **propiedad** (ownership): el hilo que bloquea el mutex es el único autorizado por el sistema para desbloquearlo. En la práctica 2, el mutex se asegura de que la instrucción `queue[in] = dato;` y la actualización del índice `in = (in + 1) % SIZE;` actúen como un solo bloque indivisible. Si un consumidor intenta leer al mismo tiempo que el productor está escribiendo, el mutex obliga al consumidor a esperar, erradicando cualquier condición de carrera.

2.2. Los Semáforos (Señalización y Conteo de Recursos)

Los semáforos, también introducidos por Dijkstra, no son cerrojos, sino contadores atómicos acoplados a una cola de suspensión del sistema operativo. Su objetivo principal no es la exclusión mutua, sino la **señalización** y la gestión de la disponibilidad de recursos.

Internamente, un semáforo consta de un número entero y una lista de tareas bloqueadas. Posee dos operaciones atómicas fundamentales:

- **Wait:** Si el valor del contador es mayor que 0, lo decrementa en 1 y permite que el hilo continúe inmediatamente. Si el valor es 0, significa que no hay recursos disponibles; el hilo actual es sacado del procesador y colocado en la lista de espera del semáforo (duerme).
- **Post:** Incrementa el contador en 1. Si hay hilos dormidos en la lista de espera de este semáforo, el sistema operativo despierta a uno de ellos.

En la práctica, se implementaron dos semáforos simétricos para orquestar la disponibilidad de la cola circular:

- **sem_empty**: Inicializado en 10 (tamaño total de la cola). Cuantifica los espacios vacíos donde es seguro escribir. El productor realiza un *wait* sobre él, asegurándose de dormir si el búfer se llena.
- **sem_full**: Inicializado en 0. Cuantifica los elementos listos para ser leídos. Los consumidores realizan *wait* sobre él, garantizando que jamás extraerán basura de una cola vacía.

Lo interesante de este diseño radica en cómo ambos semáforos trabajan juntos: cuando el productor inserta un dato (consumiendo un espacio vacío), hace un *post* a **sem_full** para notificar a los consumidores. Inversamente, cuando un consumidor extrae un dato, hace un *post* a **sem_empty** noticiando al productor de la apertura de un nuevo hueco.

```

1 void* consumer(void* arg)
2 {
3     //saber que consumidor es
4     int id = *((int*)arg);
5
6     while (1)
7     {
8         sem_wait(&sem_full); // espera a que haya > 0
9                               productos para consumir
10
11        //bloquear mutex
12        pthread_mutex_lock(&mutex);
13
14        // consumir
15        int dato = queue[out];
16        printf("Consumidor %d extrajo: %d de posicion %d\n",
17              id, dato, out);
18
19        // sumar indice de extracci n
20        out = (out + 1) % SIZE;
21
22        pthread_mutex_unlock(&mutex); // desbloquear mutex
23
24        sem_post(&sem_empty); // +1 lugares libres!
25    }
26    return NULL;
27 }
```

Listing 1.1. Fragmento de código: Interacción del Mutex y Semáforos

3. Análisis de Resultados: Asimetría en el Consumo

Durante la etapa de experimentación y prueba, se identificó un comportamiento consistente: el **Consumidor 1** extrae una cantidad de datos significativamente mayor que el **Consumidor 2**.

Desde un enfoque teórico, dado que ambos consumidores ejecutan secuencias idénticas, se podría anticipar una distribución simétrica. Sin embargo, en la práctica de sistemas operativos (particularmente en implementaciones del núcleo de Linux), este desbalance es un comportamiento esperado debido a las políticas del **Planificador (OS Scheduler)**:

3.1. 1. Ausencia de Justicia (Fairness) en los Semáforos POSIX

Cuando el Productor invoca la instrucción `sem_post(&sem_full)`, el sistema operativo debe despertar a uno de los hilos suspendidos en el semáforo. La implementación de POSIX no mantiene una política estricta de ordenamiento FIFO (First-In, First-Out). El núcleo toma la decisión que resulte de menor costo computacional, careciendo de *fairness* o equidad garantizada para los hilos que llevan más tiempo esperando.

3.2. 2. Penalización por Cambio de Contexto y Afinidad de Caché (Cache Affinity)

Cuando un Consumidor entra en ejecución, el núcleo carga en sus memorias caché (L1/L2) las instrucciones y el contexto del hilo. Si el Consumidor 1 finaliza su ciclo de lectura rápidamente, el planificador se enfrenta a una decisión técnica: volver a agendar al Consumidor 1 (cuyos datos ya están en la caché) o pausarlo, limpiar su estado y despertar físicamente al Consumidor 2. Este último proceso, denominado *Context Switch*, es sumamente costoso. Para maximizar el rendimiento general de la CPU, el sistema prefiere la afinidad de caché (mantener ejecutando al hilo activo), lo que provoca el acaparamiento momentáneo.

3.3. 3. Inanición por Brevedad de la Sección Crítica

La sección crítica protegida por el mutex consta de instrucciones casi instantáneas (asignación de arreglos y cálculo modular). El Consumidor 1 a menudo adquiere el mutex, procesa el dato, libera el mutex e inicia la siguiente iteración llegando a `sem_wait` antes de que el sistema operativo haya finalizado la interrupción necesaria para despertar al Consumidor 2. Esto resulta en una breve pero contundente inanición (starvation) para el segundo hilo.

4. Conclusión

El desarrollo de esta práctica demuestra una solución robusta al paradigma Productor-Consumidor. El empleo de **semáforos** demostró ser un mecanismo infalible de notificación (evitando subdesbordamientos o sobreescrituras), mientras que el **mutex** aseguró el encapsulamiento atómico de la cola circular y sus punteros, eliminando el riesgo de corromper la memoria compartida.

Por otro lado, el desbalance de procesamiento entre los consumidores resulta en una valiosa lección arquitectónica: un programa concurrente matemáticamente seguro (libre de *deadlocks* e interferencias) no es inherentemente un programa

justo a nivel de recursos. El comportamiento de los hilos está profundamente dominado por las heurísticas de afinidad y optimización del núcleo del sistema operativo. Escribir software verdaderamente equilibrado requeriría una sobreingeniería que impondría un enorme impacto en el rendimiento.